

ESTRUTURA DE DADOS - AVA I

Lista Simplesmente Encadeada

Disciplina: Estrutura de Dados

Aluno: Thiago Barbosa da Silva Bastos

Introdução

O presente trabalho tem como objetivo apresentar a implementação de um programa em linguagem C para o gerenciamento de uma Lista Simplesmente Encadeada, conforme solicitado no exercício de Estrutura de Dados.

Listas encadeadas são estruturas de dados dinâmicas fundamentais na computação, permitindo o armazenamento de elementos de forma sequencial, mas não contígua, na memória. O desafio proposto consiste em desenvolver um sistema que permita ao usuário realizar as quatro operações elementares de manipulação de dados (conhecidas como CRUD: Create, Read, Update, Delete), que neste contexto são:

- Inclusão: Adicionar um novo elemento à lista.
- Consulta: Buscar por um elemento específico.
- Alteração: Modificar o valor de um elemento existente.
- Remoção: Excluir um elemento da lista.

Além destas, o programa implementa a funcionalidade de listar todos os dados armazenados, requisito central da atividade.

A solução foi desenvolvida priorizando a modularidade, a clareza do código e o correto gerenciamento de memória. Para isso, o ponteiro de início da lista foi mantido como uma variável local na função main, e a manipulação da lista foi encapsulada em funções dedicadas.

Um destaque técnico da implementação é o uso de "ponteiros para ponteiros" (No `**lista`) nas funções que alteram a estrutura da lista (como incluir e remover). Esta abordagem permite que o ponteiro de início da lista seja modificado de forma segura dentro das funções, evitando o uso de variáveis globais e tornando o código mais robusto.

A seguir, será apresentado o código-fonte completo e comentado que implementa a solução descrita.

Codigo-Fonte

```
#include <stdio.h>
#include <stdlib.h>

/// Cada nó contém um dado (int) e um ponteiro para o próximo
nó.
typedef struct No {
    int dado;
    struct No *proximo;
} No;

// Incluindo um novo elemento na lista
void incluir(No **lista, int valor) {
    No *novo = (No *)malloc(sizeof(No));
    if (novo == NULL) {
        printf("Erro: Falha na alocação de memória.\n");
        return;
    }

    novo->dado = valor;

    novo->proximo = *lista;

    *lista = novo;

    printf("Valor %d incluído com sucesso no início da
lista.\n", valor);
}

// Imprimindo todos os elementos da lista
void listar(No *lista) {
    No *temp = lista;

    if (temp == NULL) {
        printf("Lista vazia.\n");
        return;
    }

    printf("Elementos da lista: \n");
    printf("INICIO -> ");
```

```

    while (temp != NULL) {
        printf("%d -> ", temp->dado);
        temp = temp->proximo;
    }
    printf("NULL\n");
}

// Buscando um elemento na lista
No* consultar(No *lista, int valor) {
    No *temp = lista;

    while (temp != NULL) {
        if (temp->dado == valor) {
            return temp; // Encontrado
        }
        temp = temp->proximo;
    }
    return NULL; // Não encontrado
}

//Alterando o valor de um nó na lista
void alterar(No *lista, int valorAntigo, int valorNovo) {
    No *noParaAlterar = consultar(lista, valorAntigo);

    if (noParaAlterar != NULL) {
        noParaAlterar->dado = valorNovo;
        printf("Valor %d alterado para %d com sucesso.\n",
valorAntigo, valorNovo);
    } else {
        printf("Erro: Valor %d nao encontrado na lista para
alteracao.\n", valorAntigo);
    }
}

// Removendo um elemento da lista
void remover(No **lista, int valor) {
    No *temp = *lista;
    No *anterior = NULL;

    // Caso 1: O nó a ser removido é o primeiro (início da
lista)
    if (temp != NULL && temp->dado == valor) {
        *lista = temp->proximo;
        free(temp);
    }
}

```

```

        printf("Valor %d removido com sucesso.\n", valor);
        return;
    }

    // Caso 2: O nó está no meio ou fim da lista
    while (temp != NULL && temp->dado != valor) {
        anterior = temp;
        temp = temp->proximo;
    }

    // Se temp é NULL, o valor não foi encontrado
    if (temp == NULL) {
        printf("Erro: Valor %d nao encontrado na lista para
remoção.\n", valor);
        return;
    }

    // Se encontrou, o 'anterior' deve "pular" o 'temp'
    anterior->proximo = temp->proximo;
    free(temp); // Libera o nó 'temp'
    printf("Valor %d removido com sucesso.\n", valor);
}

// Liberado toda a memória alocada para a lista
void liberar_lista(No **lista) {
    No *atual = *lista;
    No *proximo;

    while (atual != NULL) {
        proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }
    *lista = NULL; // Define o início da lista como NULL
}

/**
 * @brief Exibe o menu de opções.
 */
void exibir_menu() {
    printf("\n--- Gerenciador de Lista ---\n");
    printf("1. Incluir elemento (no inicio)\n");
    printf("2. Listar elementos\n");
    printf("3. Consultar elemento\n");
}

```

```

    printf("4. Alterar elemento\n");
    printf("5. Remover elemento\n");
    printf("0. Sair\n");
    printf("Escolha uma opcao: ");
}

// Função principal
int main() {
    No *lista = NULL; // Inicializa a lista como vazia
    int opcao, valor, valorAntigo, valorNovo;
    No *resultado = NULL;

    do {
        exibir_menu();
        scanf("%d", &opcao);

        switch (opcao) {
            case 1: // Incluir
                printf("Digite o valor a ser incluído: ");
                scanf("%d", &valor);
                incluir(&lista, valor);
                break;

            case 2: // Listar
                listar(lista);
                break;

            case 3: // Consultar
                printf("Digite o valor a ser consultado: ");
                scanf("%d", &valor);
                resultado = consultar(lista, valor);
                if (resultado != NULL) {
                    printf("Valor %d encontrado na lista\n", valor, (void*)resultado);
                } else {
                    printf("Valor %d NAO encontrado na\n", valor);
                }
                break;

            case 4: // Alterar
                printf("Digite o valor antigo: ");
                scanf("%d", &valorAntigo);
                printf("Digite o novo valor: ");

```

```
        scanf("%d", &valorNovo);
        alterar(lista, valorAntigo, valorNovo);
        break;

    case 5: // Remover
        printf("Digite o valor a ser removido: ");
        scanf("%d", &valor);
        remover(&lista, valor);
        break;

    case 0: // Sair
        printf("Encerrando o programa. Liberando
memoria...\n");
        liberar_lista(&lista);
        break;

    default:
        printf("Opcao invalida. Tente novamente.\n");
    }
} while (opcao != 0);

return 0;
}
```